

dataQ 자동 표준화 첫 구현과 버그들

시점: 2026-03-30

구현을 돌리자마자 나온 문제

분석 파이프라인(일지 02)의 구현을 마치고 테스트를 22건 정도 작성해서 돌렸다. 최종적으로 전부 통과했지만, 통과하기까지 고쳐야 했던 버그들과 테스트로 고정해 둔 규칙들이 이 일지의 내용이다. 테스트 프레임워크는 JUnit 5, 대상은 `DataStandardController.analyzeTerms()` 와 `registerTermsBatch()`.

상태 판정 버그 (BUG-001)

가장 먼저 잡힌 건 상태 판정 로직이었다. 분석 파이프라인의 7단계에서 결과 상태를 AUTO / PARTIAL / FAILED 중 하나로 정하는 부분인데, 처음 구현은 이랬다.

토큰 중 NEW(미등록)가 하나라도 있으면 → PARTIAL
아니면 → AUTO

빈 토큰 리스트와 전체 NEW를 구분하지 않고 있었다. 그래서 "안되는용어"나 "핸드폰지갑명"처럼 **모든 단어가 미등록**인 경우에도 PARTIAL로 판정됐다. PARTIAL은 "일부만 고치면 된다"는 의미인데 실제로는 손덜 게 너무 많은 상태라서 UX상 FAILED로 떨어뜨려야 했다.

수정은 조건 하나를 추가하는 것이었다.

wordAnalyses가 비어있거나 MATCHED가 하나도 없으면 → FAILED
아니면 NEW가 있으면 → PARTIAL
아니면 → AUTO

이 버그를 테스트로 고정해두는 과정에서 한 가지 결가지 버그도 같이 나왔다. FAILED 상태일 때 UI에서 "승인" 버튼만 노출되어 사용자가 할 수 있는 동작이 없었다. 수정은 "FAILED도 PARTIAL과 동일하게 [수정] 버튼을 노출한다"로 해결. DSTermRecommend.vue의 액션 컬럼을 고쳤다. 두 건 모두 E2E 시나리오 테스트로 남겼다.

영문약어 조합의 경계 조건

영문약어를 `_`로 조합하는 `joinAbbrevs` 같은 로직을 테스트하다가 첫 번째 토큰이 빈 문자열일 때 구분자가 앞에 붙는 문제를 만났다. 예를 들어 "알수없음 + 번호" 케이스에서 첫 단어 `알수없음`의 영문약어가 없으면 `_NO`라는 출력이 나오고, 테스트는 `NO`를 기대했다. 이 경우엔 구분자 없이 시작하는 쪽이 정상 동작으로 봤다. 빈 토큰은 건너뛰고, 그 다음 토큰이 첫 번째처럼 취급된다.

이 규칙은 "왜 그렇게 정해졌는가"를 기록해두지 않으면 나중에 누가 봐도 버그처럼 보인다. 그래서 테스트 이름에 "빈_약어는_건너뚬"이라고 박아두고 주석에 근거를 남겼다.

TB_WORD_DICT 추천의 경로

테스트를 돌리면서 미등록 단어에 대한 추천이 너무 빈약하다는 걸 느꼈다. "핸드폰"을 넣으면 TB_WORD에는 없으니 NEW로 떨어지는데, 사용자에게 "빈 칸을 채우세요"라고만 하면 아무도 채우지 않는다. 여기서 TB_WORD_DICT(추천 사전) 테이블을 본격적으로 쓰기 시작했다. 시드로 337건을 먼저 깔았다.

TB_WORD_DICT의 역할은 이렇다. 분석 중 NEW 단어가 나오면 TB_WORD_DICT를 조회해서 "핸드폰 → HNDPHN / HANDPHONE"처럼 관례적 영문약어·영문명을 끌어다 결과의 `newWord` 필드에 미리 채운다. 사용자는 이 추천값을 보고 그대로 승인하거나 수정한다.

TB_WORD_DICT 조회는 테스트에서 세 건으로 검증했다. 사전에 있는 단어는 값이 나오고, 없는 단어는 null이 나오고, 추천값이 결과 VO의 올바른 필드에 세팅되는지. 지금 보면 너무 당연한 테스트인데 이 중 "NewWordSuggestion 필드 세팅"에서 한 번 필드명 오타로 실패해서 잡았다.

선등록 순서 문제

`registerTermsBatch` 에서 미등록 단어(`newWords`)를 받아서 등록하는 로직이 있다. 이 로직은 순서 의존이 있다.

1. `newWords` → TB_WORD INSERT (각각)
2. 반환된 새 `wordId`들을 `items[.].words[.].wordId` null 자리에 매핑
3. TB_TERMS INSERT + TB_TERMS_WORDS INSERT

처음 구현에서는 2단계(매핑)의 인덱스 처리가 불완전해서, 같은 트랜잭션 안에 있어도 새 `wordId`가 `items`에 제대로 반영되지 않아 FK 참조 에러가 났다. 테스트로 `newWordIdMap[0] → uuid-new-word-1` 매핑을 검증하는 건을 추가하면서 고쳤다.

또 하나, "NEW 단어 필수필드 누락 시 등록 불가" 건은 테스트는 작성했는데 이 시점의 구현은 빈 영문약어로도 등록을 시도하고 있었다. 프론트/백엔드 양쪽에 검증을 추가해야 한다고 적어놓고 다음 작업으로 미뤘다.

테스트 전체 통과, 그러나 남은 과제

테스트 22건이 전부 통과했다. 실행 시간 0.057초. 숫자로만 보면 "자동 표준화 추천 시스템의 1차 구현이 완료됐다"고 보고할 만한 지점이었다. 다만 테스트 결과 문서 말미에 적어둔 "미구현 사항" 네 줄이 다음 작업의 출발점이 된다.

항목	상태	해야 할 일
NEW 단어 필수필드 검증	빈 영문약어로도 등록 시도됨	프론트/백엔드 검증 추가
단어 선등록 흐름	registerTermsBatch에서 한 번에 처리	수정 다이얼로그에서 단어 등록 선행을 명시
selectCoveringTokens 개선	OKT 토큰만 사용	TB_WORD 기반 greedy longest-match 추가
영문약어 스코어링 방향	짧을수록 가산	길수록 가산으로 뒤집을지 검토

세 번째 줄이 이 일지에서 가장 중요하다. 첫 구현에서는 한글명을 단어 단위로 쪼갤 때 OKT 토큰만 사용하고 있었다. OKT가 뽑아준 토큰을 TB_WORD에서 찾아보고, 없으면 NEW로 떨어뜨리는 구조. 문제는 OKT가 뽑아주는 토큰이 **TB_WORD의 구성 단위와 일치하지 않는다**는 것이다. 예를 들어 TB_WORD에 사용자, 이름 이 있고 한글명이 "사용자이름"으로 들어오면 OKT는 보통 사용자 + 이름 으로 잘 쪼개는데, "기준일자" 같은 건 기준일자 한 덩어리로 뽑기도 하고, 복합명사는 더 심각해진다.

해결 방향은 테스트 결과 문서에 적어놓은 대로였다. "TB_WORD 기반 greedy longest-match 추가". 현재 한글명 문자열을 왼쪽부터 훑으면서 TB_WORD에 있는 가장 긴 단어를 욕심껏 매칭한다. OKT 토큰은 보조로 쓴다.

이 한 줄이 다음 일지(04)의 출발점이 된다. greedy로 시작했다가 반례를 만나고, 국소 보정을 네 번 반복한 뒤 DP 기반 전역 최적 분리로 전환하게 되는 구간이다.

부대 결정사항

테스트를 돌리는 과정에서 몇 가지 결가지 결정도 같이 내렸다.

- mvn test 는 프론트엔드 빌드를 포함해서 느리다. 단위 테스트만 빠르게 돌릴 때는 mvn surefire:test -Dincludes="**/TermRecommendTest.java" 로 직접 surefire만 실행한다. 이걸 개발 가이드에도 적어뒀다.
- q-common을 고치면 jar로 빌드한 뒤 q-center의 lib/에 복사해야 q-center가 변경을 인식한다. 이 순서를 한 번 놓쳐서 "수정이 반영되지 않는" 상태로 한참 원인을 찾았다.
- 테스트 데이터는 TB_WORD 7건(사용자·이름·전화·번호·주소·운전·면허), TB_WORD_DICT 2건(핸드폰·지갑)으로 최소화. 시드 데이터가 커지면 테스트 기대값을 유지하기 어려워진다.